

type coercion — Implicit [JS conversion by default]
 — Explicit [manually coercion]

papergrid

Date: / /

Type coercion

① What is abstract operation? — They are spec-only helper function, the JS spec uses to define behaviour, eg - ToNumber, ToBoolean, etc. They are not exposed to JS directly, but you can approximate them;

Types of Abstract Operation → (1) ToBoolean(arg) :-

Conversion of any datatype in to boolean (T/F);

m.f.m.p Falsy Value :- +0, -0, NaN, undefined, false, "" (empty string), Null;

rest all are truthy value { 1, 2, symbol can consider as truthy, etc };

ToBoolean() you can't use directly, If you want to check anything is true/false. You can use 'logical not operator' { ! } similar but it return opposite of value;

eg- !(" ") → true (since " " → falsy value) | !(false) → false, etc.
 !(+0) → true | !(-0) → true | !("0") → false

② ToNumber(arg) :- The abstract operation ToNumber() converts arg to a value of type Number;

(*) Undefined → NaN (converted value)

(*) Null :- +0

(*) Boolean :- T → 1, F → +0

(*) Object :- Step 2 → 1st + ToPrimitive()
 2nd + ToNumber()

Number :- no conversion

String :- See grammar for ^{conversion} algorithm

Symbol :- throw a TypeError exception;

eg → 1 - '2' ⇒ 1 - 2 ⇒ (-1)

(conversion to Num { sub. method called ToNumber() }
 for both 1 & 2 (operands);

Primitive Data Types :- Those DT. which are atomic in nature (further dividable into parts) eg → number, etc;

Non Primitive DT :- Those which ^{are} created by using Primitive DT.
 eg → obj, array, etc;

(+) operator any one of L/R is String
then concat
otherwise addition;

papergrid

Date: / /

Primitive value - Number, String, undefined, null, boolean;

ToPrimitive() Convert into primitive number;

(+) - If any one is string $\rightarrow$ Concat; $\{1 + '2' \Rightarrow 12\}$

else $\rightarrow$ convert into number (ToNumber()) &
add; $\{1 + 2 \Rightarrow 3\}$

$\rightarrow$ before this check L/R is primitive() then only
otherwise $\rightarrow$ It behave different;

eg $\rightarrow [1, 2] + [3, 4] \Rightarrow [1, 23, 4]$

eg $\rightarrow 1 + \text{null} \Rightarrow 1 + (+0) = 1$

$1 + \text{undefined} \Rightarrow$ convert into ^{Number} ~~String~~ $\rightarrow 1 + \text{NaN}$
 $\Rightarrow (\text{NaN})$

(anything) + NaN = NaN

Abstract Equality Comparison (==) & Strict Equality Comparison (===)

(i) Abstract E.C. (==) - $x == y$, x, y are values, produces T/F.

Step 1:- if $\text{Type}(x)$ is same as $\text{Type}(y)$ then do Strict equality comparison
(Implicit Conversion) ($===$)

Otherwise do these operation: -

(i) if $x == \text{null}$ & y is undefined $\rightarrow$ true

$[\text{null} == \text{undefined}] \Rightarrow \text{true};$
or $[\text{undefined} == \text{null}]$

(ii)

$\uparrow$
String

number

$\rightarrow$ Convert into Number ToNumber();

NaN == NaN => false,
 NaN == NaN => false;
 NaN == Undefined => false
 typeof NaN

(iii) $\frac{L}{\text{Boolean} \rightarrow \text{ToNumber() convert}}$
 $(T) \rightarrow 1, (F) \rightarrow 0$

(iv) $\frac{L}{\text{String/Num/Symbol}} == \frac{R}{\text{Object}}$
 Same for swap version; $\hookrightarrow$ Convert into Primitive;

Strict Equality: If types are diff return false;
 else: - if L/R is NaN $\rightarrow$ return false;
 eg $\rightarrow x = +0 \mid y = -0$
 $[x === y] \rightarrow \text{true}$
 eg $\rightarrow$

New video of type casting: - $\rightarrow$ ToPrimitive: - It take two diff args -

(i) input arg, (ii) optional arg [preferredType];

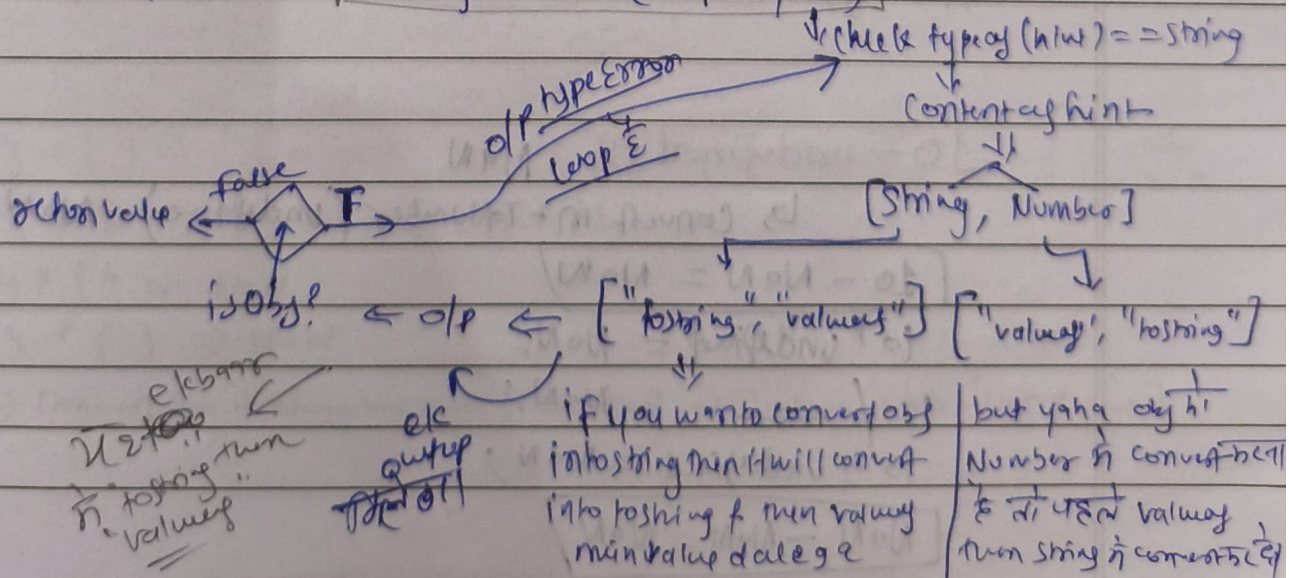
It converts your object into non object and if this value is capable of converting to more than one primitive types, it may use the optional hint preferred type to favour that type;

(i) If preferred type not define by default $\rightarrow$ default:

String ——— 'String' | Same for Number;

(ii) If your hint is default then make it number;

(iii) Now call $\rightarrow$ `OrdinaryToPrimitive(input, hint)`



let x = {};

x.toString() → '[object object]'

or x.valueOf() → {} + return default 'x {object}'

Imp

Example:-

x = 10;

y = 20;

x - y = NaN? ~~fit~~

(→) do ToNumber(x/y)

Internally checks any one is obj? Yes (Y)

With hint = number

ToPrimitive(Y)

[valueOf, toString]

valueOf(Y) = Y ⇒ toString(Y) = '[object object]'

but this is not a number

again call →

with hint = string

toString() → + some

valueOf('[object object]')

NaN

80M = [10 - NaN] = NaN

⇒ NaN

Imp

If you override the toValue | toString method then it behaves differently.

⇒

10 - undefined = NaN

↳ convert to ToNumber(undefined) = NaN

10 - NaN = NaN

10 * undefined = NaN;

10 / — = NaN;

10 + undefined = NaN;

NaN - NaN = NaN

$\text{typeof}(\text{null}) \Rightarrow \text{Object}$

papergrid

Date: / /

1) $10 - \text{null} = 10$

$\hookrightarrow \text{ToNumber}(\text{null}) = +0$

$\text{null} - \text{null} = 0$

$\text{null} * \text{null} = 0$

$\text{null} / \text{null} \Rightarrow \text{undefined} \rightarrow \text{NaN}$

why $\Rightarrow \text{null} / \text{null} = (0/0) \Rightarrow \text{Error} [\text{undefined}]$

$\downarrow \text{ToNumber}(\text{undefined})$

$\downarrow$

NaN

2) $\text{null} + \text{NaN} = \text{NaN}$

3) $\text{undefined} + \text{null}$

$\downarrow \text{NaN} + \text{null} \Rightarrow \text{NaN}$

4) $[2, 3] + [4, 5]$

$\downarrow$

2, 34, 5

toString[2,3]

$\downarrow$
'2,3' + '4,5'

$\downarrow$

[2,34,5]

5) $[2, 3] - [4, 5] = \text{NaN}?$

ToNumber $\downarrow$ obj

$\downarrow$ obj

ToPrimitive([2,3])

$\downarrow$ Number

Value of ([2,3]) = ?

[2,3] $\Rightarrow$ toString() $\Rightarrow$ '[object Object]'

$\hookrightarrow$ toString() $\Rightarrow$ (2,3) return

'2,3' $\Rightarrow$ ToNumber('2,3')

$\downarrow$

NaN (because of comma)

6) function f() { } $\Rightarrow$ value of() $\Rightarrow$ return fx if it is

$\hookrightarrow$ toString() $\Rightarrow$ return fx as string format

Ex: 'str' + fx $\Rightarrow$ [strfx] $\Rightarrow$ 'strfx'

7) $\{ \} + \{ \} \Rightarrow$ '[object Object][object Object]'

$\{ \} + [1, 2] \Rightarrow$ '[object Object]1,2';

$\{ \} * \{ \} \Rightarrow \text{NaN}$

$\{ \} * [] \Rightarrow \text{NaN}$

$\hookrightarrow$ convert into Number $\Rightarrow$ '[ob ob]' $\Rightarrow$ ToNumber $\Rightarrow$ (NaN * NaN)

Q. Comparison Operator: —

$\downarrow$ ~~3 < 2~~ $\rightarrow$ $1 < 2 < 3 \rightarrow$ True?
 $\downarrow$ ~~2 < 3~~ $3 > 2 > 1 \rightarrow$ false?

How Comparison Operator works [It always return boolean Number]

$\downarrow$ (left $\rightarrow$ right) {operator precedence}

Ex — $1 < 2 < 3$

$\downarrow$ [1, 2] + [2, 3]

$(1 < 2) \Rightarrow \text{op} \Rightarrow (\text{True}) \Rightarrow \text{ToNumber}(\text{True}) = 1$

$(1 < 3) \Rightarrow$ True

Ex — $3 > 2 > 1$

? $\text{op} = [2, 1] - [1, 2]$

$(3 > 2) \Rightarrow \text{op} \Rightarrow \text{True} \Rightarrow \text{ToNumber}(\text{True}) = 1$

$(1 > 1) \Rightarrow \text{op} =$ false

Q. let y = 10;

'Hello value of y is 10';

$\rightarrow$ Type Coercion
Conversion into String

$\downarrow$ op

Hello value of y is 10

let y = {y};

then $\rightarrow$ [Hello value of y is [object object]]

why

$\text{tostring}(\{y\}) \Rightarrow$ [object object] $\rightarrow$ String() / toString()

[10 $\Rightarrow$ by default Number] $\downarrow$ 2nd operation;

Same

Q. S = "123"

$\text{num} = +S \Rightarrow 123 \Rightarrow$ Number — the num ber, Conversion;

Q. S = "123"

$\text{bool val} = !!S \Rightarrow$!(!)

! false = True

$\downarrow$
for boolean Conversion
[] True
> ! True = false

Undefined Vs Not defined: - variable does not exist;

actual variable exist
but value not init (or consuming before init)

papergrid

Date: / /

Manual type conversion :-

(1) Number("123") $\Rightarrow$ 123
("") $\Rightarrow$ 0
("0") $\Rightarrow$ 0

(1) String(10) $\Rightarrow$ '10';
String(0) $\Rightarrow$ '0';
String(-0) $\Rightarrow$ '0';

(+0, -0) but ux case bahut $\bar{e}$ (game wale field mein su)

(acceleration + (-))

Ques:-

W.A function which take a value and return whether a value is NaN or Not? $\Rightarrow$ but you can't able to compare [NaN == NaN]? Yes then how?

typeof(NaN) = ? Number

isNaN()

Number.isNaN()

it check

$\hookrightarrow$ it strictly check;

anything if last conversion is NaN then true (ex-string)

$\Rightarrow$ $-0 == 0 \Rightarrow \text{True}$, $-0 === 0 \Rightarrow \text{True}$ | $-0 > 0$ | $-0 < 0$
then how we know its is $-0/0$ { false }
[Object.is(-0, 0) $\Rightarrow$ false]
Math.sign(-0) = -0
(0) = 0

AutoBoxing :- let x = 10; x.toString() $\Rightarrow$ "10"

$\hookrightarrow$ but this is allow him obj (present)

So In JS there is a concept called autoBoxing?

JavaScript [Bool, BigInt, Number, Undefined, Null, Symbol, String]

papergrid

Date: / /

For a moment JS ^{or wrapped} convert primitive value (variable) into object
so that you can use that method which ~~is~~ only present
in obj. eg → toString(), valueOf(); This is ~~aka~~ autoboxing;

But it is not applicable with primitive literals (except string);

eg →

10.toString() X

(10).toString() ✓

"abc".toString() ✓

"abc".toLocaleString() ✓

[Using string literals you can
use both of them]

Whenever we try to access a method or property on a primitive, that
primitive is wrapped into an object. That is called autoboxing.

(M.Dug)

Lee—

Asy = Await (Async functionality)

Introduction to Callbacks :-

Those function which can ^{accept} take function as an argument, is
Kla Hof (Higher order function).

there-~~fx~~ which is being passed ^{into the} ~~through~~ as an argument one k/a
Callback function;

Callback fx :- These are those function which are passed as an argument to a Hof;

eg → `arr.map(fn(e));`
 ↓ ↓
 HOF callback function;

Benefits of Callbacks:- We can decide on the runtime what to send as an 'implementation' of callback.

Disadvantages of fallback: (1) Call back hell (minor dis-adv) [readability problem]

ii) Invasion of Control (major dis-adv);

```

17) Callback hell: — f1(function()) {
    f2(function()) {
        f3(function()) { }
    }
}

```

When one HOF calling (arg) another fx that fx again call callback fx
and this callback fx again calling another callback fx.
Then creates deeply nested, hard-to-read, and hard to maint
this is nty but callback-hell;

2) Inversion of Control :-

eg- 'Super Pay' { payment to merchant }

↓
SDR

nightcode.io

Suppose you make payment (credits, function $f(t)$)

here we are taking a bust on paper pay [book 4]

Here we are building a trust with superpay that only one time payment will happen, it will not re/execute multiple times, it can not failed make payment for; α of transaction

What if it does not follow these things [so mix ntg but Inversion of control] we are handed over to me another fx that it will call our implementation; $\hookrightarrow$ the control to another fx;

Inversion of control is a phenomenon in which we give the control of executing our code back to some other HoF which may or may not be calling it right way.

Asyn Programming:- There are two ways to write a program -

(1) Synchronous Program

(11) Asynchronous Buoys

line by line Execution

same line by line but

(pure execution based
new execution)

[if Line-1 is not fully executed but Line-2 started executing, etc;]

[Quinn King]

do not depend upon virus execution

(not sequential order of execution)

Q. What is the default nature of JS?

Any native piece of JS code is always synchronous native code (code which is defined in documentation)

JS is sync in nature bcoz of single threaded in nature;

In single core CPU:- two or more than two process can't be executed parallelly;

It can execute only one process at a time;

↳ runtime features;

Asyn code:-

```
setTimeout(function () {
```

```
  console.log("Time up")
```

```
}, 10);
```

```
console.log("Hello world")
```

Here it behave as asyn why?

bcoz setTimeout() is not a native piece of JS code;

How this type of execute (behind the scene) is:-

runtime env provides → event loop, queue (microtask queue, macro-task queue), call stack

event loop checks whether call stack is free or not, if it is free then execute the non-JS-native code;

Asyn JS and its features

Closure properties:-

eg-

```
function fx() {
```

```
  let a = 10;
```

```
  return function() {
```

```
    console.log(a)
```

```
  }
```

```
}
```

```
const o1 = fx()
```

o1() → 10 (print) how?

console.dir(o1) → former default

you will get scope →

- Cloning []
- script
- Global

remember those variable which we write

def:-

Cloning is a mechanism through which any memory remembers all those variables which are defined in outer scope; (outer scope)

Q How to resolve the problem of Callbacks? —

Solⁿ:-

Promises in JS (official part of JS)

↓
Solve both problem;

but technically it can't resolve callback hell (promises hell)

but, fully resolved too;

Promises:- are readability enhancers. They are special JS object which can help us to control future related tasks.

1) How to create a promise?

(ii) How to consume a promise;

NOTE:-

Generally we prefer using promises when we have to deal with future task;

Problem:-

Implement a set of dummy functions which can mimic the behaviour of the following functions:-

(i) download:- This function should mimic downloading of some content from a url;

(ii) writefile:- This fn should mimic writing the downloaded content to a file;

(iii) Upload:- This function should mimic uploading the file to a server;

Now after you've implemented these functions, try to use them in a scenario where we first download a file, then write it to a disk and then upload it to the server;

optional chaining)

P. → checks if it is present then do otherwise skip eg —
callback? (promise)
→ check whether passed or not;

papergrid

Date: / /

- (1) After downloading/writing/uploading we need to call a callback so that we can able to perform some operation eg → after downloading we have some operations to perform with this data so we can pass cbfr and called after completion of downloading file;

```
function (url, callback) {  
  console.log("data is downloading from this url", url);  
  setTimeout(() => {  
    const console.log("data are downloaded");  
    callback(data);  
    // 3, 3000  
  }, 3000);  
}
```

```
function writeFile(data, filename, callback) {  
  console.log("write data in " + filename);  
  setTimeout(() => {  
    console.log("writing to a file is done");  
    status = true callback();  
    // status  
    // 3, time  
  }, time);  
}
```

```
function uploadFile(data/filename, url, callback) {  
  console.log("uploading file");  
  setTimeout(() => {  
    console.log("file uploaded");  
    status = true callback();  
    // status  
    // 3, time  
  }, time);  
};
```

```
function process(url, (data) => {  
  writeFile(data, "file.txt", (status) => {  
    uploadFile("file.txt", url, (status) => {  
      console.log("All done");  
    });  
  });  
});
```

How to solve $\rightarrow$ promise $\rightarrow$

In JS we have a promise construct;

this construct expects a callback in the arguments:

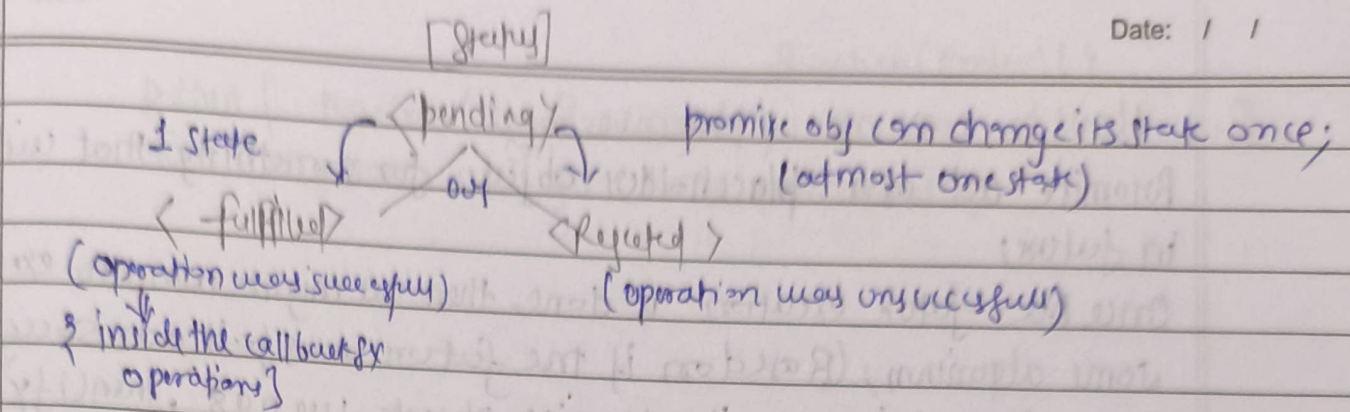
* Why is it called executor callback? $\Rightarrow$ bcoz when we create a promise object at that point of time our constructor execute this callback; That means: this cb. is executed by the promise constructor immediately;

this is also cdfx;

eg:- `const p1 = new Promise((resolve, reject) => {
 resolve("promise has been resolved");
});`

(promise has three states)
At a time only one state;

pending:- whenever a promise obj is created (by default pending state)



Operation refers to an algorithm written inside a function (cb);

resolve() → If you called this fx inside cb → Status: fulfilled;
 reject() → → Status: Rejected;

Value: PromiseValue (v:Emp) : — Initial Value: Undefined;

using resolve(), reject() you can change the promise value?

resolve(dataValue)

↓
 ↳ changing the value;
 takes an argument
 which change the value from undefined → data value;

Promise has some internal methods: — finally(), catch, then(), etc;

Some hidden properties: — onFulfillment: [], onRejected: []

Both are arrays;

Q. What is the use of these arrays?

- 1) These arrays are internally managed by Promise and initially are empty.
- 2) We can store some fx (callback) in both of these arrays, How?
- Using then() fx available on a promise object;
- 3) These function remain in their respective arrays till the time promise is pending.
- 4) If the promise moves to the fulfilled state, all the fx store in onFulfilled array is queued in a new queue in the memory called as 'microtask queue'; & onRejected array does not.
- 5) If the promise goes to rejected state then all the function in the onRejected array goes to micro-task queue & onFulfillment array does not;

Q. How to consume a promise?

Promise act as a placeholder object for something that will come in future;

Once the future execution is done, then we might want to execute some algorithm; (Based on if the future was fail or success)
To achieve this on the promise object we have, `then()`;

```
const pr = new Promise(executor);
```

```
pr.then(onFulfilled, onRejected)
```

↳ mandatory (fx) ↳ optional (fx)

this will go to the
onFulfilled array;

this will go to the
onRejected array;

eg- `pr.then(a, b)`

`pr.then(c, d)`

`pr.then(e, f)`

⇒ onFulfilled: [a, c, e]

onRejected: [b, d, f]

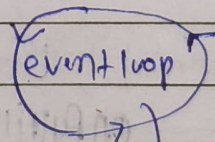
Macrotask Queue

all Callbacks timer

Microtask Queue

all those promise which are

present in onFulfilled[]
onRejected[]



priority → (microtask)_{pr} > (macrotask)_{pr}

Job promise fulfilled hojaya then push all the onFulfilled[] inside microqueue and skip the rejected array for (do nothing) and vice versa;

Event loop execute microtask first (higher priority);

Then after macrotask queue execute in (FIFO order)

A Promise fulfillment → It return (generate promise value)
you can get it from `then()` `f(promiseValue)`

[This is
not always
true!]

It is
simplified
explanation

```
function(pvalue){}
```

```
pattern { -function (promiserevalue) { }, }
```

for every thing we ^{can} get same promiserevalue;

Q. What is cutting edge technologies? Eg → GRPC

papergrid

Date: / /

Async Await

- (i) Promise.resolve(): method (static method) resolves a given value to promise. If the value is promise, that promise is returned. If the value is a 'thenable', Promise.resolve() will call then() method with two callbacks if prepared; otherwise it returned promises will be fulfilled with the value;

Eg - let x = promise.resolve(123);

x.then | (val) ⇒ { console.log(val) }

↓

123

this then() also return a promises but with val undefined
because we are not returning anything from this fx;

- (ii) Promise.reject(): - static method return a promise object that is rejected with a given reason;

promise → callbacks → microtask queue;
normal callbacks → ~~eg~~ event queue;
microtask queue has higher priority;

→ But bcoz of .then() & .catch() it also creating promise hell;
We need to resolve it, how? → (async-await)

async & await

- (i) We can declare a function async;
(ii) If you declare a function async, it does the following:-
(a) It allow the use of await keyword.
(b) It allow the consumption of a promises using await;
(c) on async fx always convert your return value to a promise;

eg- `async function fx() {
 const data = await fetch(url)
 return data;
}`

- (i) `.then()` is a method of promise. It does not block execution, it registers a callback to run when the promise is resolved; code after `.then()` keep running immediately;
 - (ii) `await` is used inside the `async fx`; It pauses the execution of that `fx` until the promise is resolved or rejected.
code after `await` runs only after the promise settles
 - (iii) The order of logs (fx-execution) can change depending on whether you use `.then()` or `await`; but if it is only sync operation then only it will block if it is async operation like creating a promise(sync) but resolving or rejecting is async operation (with `setTimeout`) at this time it will return a promise with undefined value and pending state and then start executing another line of code;
- Inside `async fx` things looks sync but overall its async;

Eg:- `function fetchData(url) {` → Sync & 4

`return new Promise( fx (su, sy) {`

`console.log("Start downloading", url);` - ②

`setTimeout( fx ProcessDownloading() {`

`let data = "dummy data";`

`console.log("Download completed");`

`resolve(data);`

`}, 7000) }`

`console.log("Start")` - ①

`let promiseObj = fetchData("url");`

`promiseObj.then( fx A(val) => console.log(val) );` - ⑤

`console.log("end")` - ③

`}`

But under the hood async await will work differently
(iterators and generator) are used under the hood;

papergrid

Date: / /

Lee-async await (2nd part)

Code:-

```
async function fn() {  
  const res1 = await call1();  
  console.log("processing data");  
  const res2 = await call2();  
}
```

When you call `fn()` what happens step by step —

i) `fn()` starts → `await call1()` → runs and returns a promise (data: undefined, status: pending)
ii) and immediately saves the rest of `fn` and exits to handle other things.

iii) Later when `call1()` resolves JS ~~mean~~ resumes the saved function with `res1`, `console.log()` → `await call2()` again returns promise and wait →

iv) when `call2()` resolves resume and finish `fn`;

Code:-

Function `fn()` {

return new Promise((res, rej) => {

setTimeout(() => {

console.log("Promise executing")

res(123)

, 5000)

});

}

{ status: , value: ,
 onfulfilled: [anon, anon]
 onrejected: [] }

let x = fn();

x.then((data) => {

console.log("value of promise is", data) // 123

}) return 200;

x.then((data) => {

console.log("value of promise is", data) // 123

})

get new promise

Date: / /
Conserved;

but if you will do $\Rightarrow x$. then $(d \Rightarrow \text{return } 100)$. then $(d \Rightarrow \text{console.log}(d))$

Here you will get 100 (not 123)

(1) What is the use of previous logic?

QW Sometime we need to do diff logic on some data then we can do without creating an extra variable (which stores first data & then ops):

Conversion to promise: — If the value is not a promise, it converts the value to a resolved promise, and waits for it. The awaited value identity does not change as long as it does not have a then property that's callable;

eg — async function fn() {
 const y = await 20;
 console.log(y); // 20 ; let obj = {5};
 console.log(await obj === obj) // true
 };
 fn();

Async-Await Hell: — eg $\rightarrow$ `const seq1 = await P1();`

`const seq2 = await P2();`

`const seq3 = await P3();`

This is k/a async await hell

How to resolve this things? $\Rightarrow$ `promise.all();`

`promise.allSettled();`

`promise.any();`

(1) `promise.all()` $\rightarrow$ method takes an iterable of promises as input and return a single promise; The return promise fulfilled when all of the i/p promises fulfill (including when an input (empty iterable) is passed), with an array of the fulfillment values;

* It rejects when any of them of the i/p promises rejects;

eg- `Promise.all([pr1, pr2, pr3, ...]).then((val) => console.log(val))`
 `.catch((err) => console.log(err));`
 ↑
 { return another promise }
 ← o/p
 array of output;
 but in order;

* Each promises executed parallelly but final res once all get resolved or any one get rejected;

1. How to do chaining:- When 3rd promise ~~data~~ is depend on 2nd prom and 2nd on 1st promise then you can simply bind in a fx and call that fx inside `promise.all([fx()]);`

eg- `const res1 = await p1();`
`const res2 = await p2(res1);`
`const res3 = await p3(res2);`
`return res3;`
 bind in a fx (async) and call that fx;

2nd Promise.any():- It also takes array of a promise and return fulfill value once any one of them get fulfilled and gets reject when everyone get rejected();

* But how do we know which promise resolved first?

3rd:- Promise.allSettled():- Same as `Promise.all()` but it will return all the res (fail/rejected or fulfilled) and it wait for everyone should be either res/settled then only it will return; (array of value (promise) you will get);

How to handle errors in JS :-

(exception)

- (i) **throw** :- The throw statement throws a user-defined exception; Execution of the current fx will stop (the statement after throw won't be executed) and control will be passed to the first catch block in the call stack. If no catch block exist among the caller fx, the program will terminate;

eg -

```
function fn() {
  if (err) {
    throw 'something went wrong';
  }
}
```

```
try {
```

```
  fn();
```

```
}
```

```
catch (err) {
```

```
  console.log(err) || console.error(err)
```

```
};
```

Parameter is not a number;

eg - `user.then(d => { console.log(d) }, fx(err) { console.log(err) })`

if err

this will call

err

`.then(d => { }, catch(err => { }));`

or

`.then(d => { }, errfx) .then().catch()`

if err

exec

↓

if err

if not

execute

no execution;

finally() method:— method of a promise schedules a function, the callback `fn`, to be called when the promise is settled. like `then()` & `catch()`, it immediately return an equivalent promise object, allowing you to chain calls to another promise method, an operation called composition;

• `finally(() => {})`
 ↳ It always execute; ✓
 either if success or rejected ✓

Error first callback:— The first arg of the callback is reserved for an error object. If an error occurred, it will be returned by the first err arg;

and same similarly second arg is reserved for any successful data; if no error occurred, err will be set to null and any successful data will be returned in the second argument.

eg- `fs.readFile('abc.txt', fn(err, data) {
 //code
 });`

↳ It's a coding style;

eg-

`function fetchData(url, fn) {`

`//processing`

`console.log(data)`

`fn(err, res);`

`}`

— here fetchData will pass

`fn(err, res) {`

`(err/data)`

`if (!err) {`

`return res`

`}`

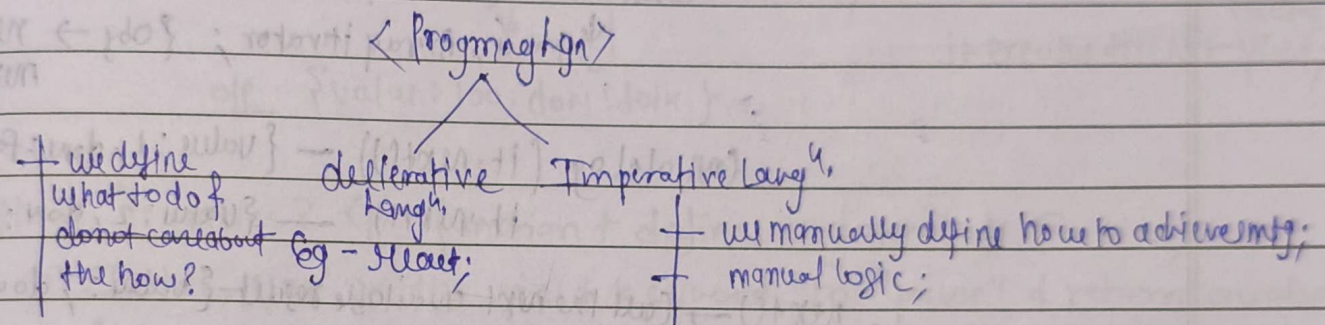
`# fetchData("url", fn) {`

(M. Imp)

Lee = Generators of Iterators (Adv Concept)

Iterators :- It is a general prog^m term (exist in multiple lang)
Ex- python, C++, etc;

- It is a design pattern. Its a behavioural design pattern that lets you traverse element of a collection without exposing its underlying representation (eg - stack, queue, etc);



eg print elements of array :-

Imperative way -

```
for (let i = 0; i < arr.length; i++) {
  console.log(arr[i])
}
```

Declarative way - Req :- we do not want to fetch all elements at a same time, we want whenever we want we can able to

get the next value [Index];

eg - function fetchNewElement(arr) { let index = 0; function returnNext() { const val = arr[index]; index++; return val; } return returnNext; }	const autoFetcher = fetchNewElement([1, 2, 3, 4]) ↓ contains 1 + x; console.log(autoFetcher) // 1 console.log(autoFetcher) // 2; ! { using closure property we are accessing index var }
--	---

How to write your own for (or mechanism) like asgn-await?

papergrid

Date: / /

you can add one edge condition — ^{Not possible}
if (index > arr.length || index < 0) {
return undefined;
}

NOTE:- This is the imperative way to create iterators;

How to use iterators: — const arr = [1, 2, 3, 4, 5]

const it = arr[Symbol.iterator]()

↓ Now, Array iterator; {obj → next() method exist}

console.log(it.next()) — {value: 1, done: false}

it.next() — {value: 2, done: false}

iterators ← next() → Call the next available result {value:, done:}
value() → it shows the next fetched value;
done() → false, means more element exist
→ true, means no element are there;

[MDN] An iterable obj is an obj that conforms to the iterator protocol by providing a next() method that return an iterator result object.

Iterable:- The iterable protocols allow javascript object to define or customize their iteration behaviour;

eg- arrays are iterable; means array follows the iterable protocol; object are not iterable directly;

How to make fully declarative "fetchNextElement"?

Soln

Using Generators; How? Generators are special function;

It can be paused and resumed, allowing you to produce (or generate) a sequence of value over time, instead of computing them all at once;

Q How to use debugger?

papergrid

Date: / /

↓ declarative approach (we are not using our logic);

eg-

```
function* newGenerator() {
  yield 100;
  yield 55;
  yield -1;
}
```

 It execute differently;
 const it = newGenerator();
 ↓
 it.next() → 100
 it.next() → 55
 ↓
 off {value: 100, done: false}
 Consider log ("Inside generator")
 yield 100;
 both will execute
 it execute till next yield;

Q How it works? <?> function * defines a generator function;

- <?> Inside, you use the yield keyword to "pause" & return a value;
 - <?> Calling the generator function returns a generator object;
 - <?> You use .next() to get the next value;
- (after the first yield it will move outside the callstack);

eg-

```
function* gen(arr) {
  for (let i = 0; i < arr.length; i++) {
    yield arr[i];
  }
}
```

 You can yield any type of data (even fx);
 const it = gen(arr);
 it.next() → 1, 2, 3, 4, ...

<?> You can also pass arg in next() [arg];
 const X = yield 10; this much only executed during
 it = fx();
 it.next() → {value: 10, done: false}
 it.next(20) → In this line X=20 assign;

const X = yield 10; → 1st
 const Y = X + 10; → 2nd
 yield Y; → 3rd
 ↳ o/p [value: 30]
 if(20) →

Generator obj is returned by a generator fn and it conforms to both the iterable protocol and the iterator class;
Generator is a subclass of the hidden 'Iterator' class

papergrid

Date: / /

i.e. can we use return keyword $\Rightarrow$ Yes;

but return 44

↳ means yield : {value: 44, done: true}

↳ true

{ yield 10; $\rightarrow$ 1st

return 44; $\rightarrow$ In 2nd iteration {v: 44, done: true}

{ yield 20; $\rightarrow$ In 3rd iteration: - {v: undefined, done: true}

↓
Last yield value; { do not pass return }

Generator with promise

function* exec() {

yield new Promise((res, rej) $\Rightarrow$ {

res(10)

})

}

const it = exec();

it.next() $\rightarrow$ {value: Promise, done: false}

↓ How to resolve;

it.next().then(); $\leftarrow$

$\Rightarrow$ If you want to execute multiple promise (and also support chaining) then you can do it easily by writing Generator;

eg - function* exec() {

const downloadData = yield download("url");

const writeData = yield writeFile(downloadData);

}

const it = exec();

it.value.next it.next.value

```
const it = it.next(); → {value: Promise, done: false}
it.value.then(function doAfterReceiving(val) {
  console.log("...", val);
  if (! it.done) return; // no yield
  const futVal = it.next(value);
  if (futVal.done) return;
  futVal.value.then(doAfterReceiving) // recursion
});
```

Above code {async-await} → Instead of this you can use async-await?

```
async function() {
  // instead of yield replace this with 'await' keyword
}
```

and no need to define iterators and next();
// declarative approach;

P.T.O

lets talk about Object / (Some things)

Important objects, Methods

(i) How to ^{get} find keys of an object: - (keys are unique in obj)
List

Code: - `Object.keys(obj1)` → array of keys; (string)

(ii) How to fetch all values of an obj? :-

Code: - `Object.values(obj1)` → `['', ...]` array of string;

(iii) How to key:value in array format: -

Code: `Object.entries(obj1)`

eg - o/p `[['name', 'xyz'], ['age', 22]]`;

(iv) How can we check the # of key:value pairs in obj?

eg - `Object.keys(obj).length`

Mutability and Immutability in JS Objects

What is mutable: - Eg - `let x = 10; x = 9;`

- Here we are able to update the value of x, this mechanism of being able to update a variable's value is called as mutability
- IF we are not able to modify the value then it is immutable;

Immutable: - `const x = 10; x = 9` (not allowed);

↳ to make them immutable;

but this is not happen with obj/array (why? becoz we internally storing array pointer value but we can update the value of array/obj;)

`const a;` (not allowed)

↳ You have to give some value;
↳ Syntax Error: Missing initializer in const declaration;

Q.7 Why do we need immutability?

There are a lot of places in programming where immutability becomes extremely important

Eg - DB connection configuration (value), most of the time (or less) we ^{do not} want to update the configuration of this DB connection. In this type of scenario immutability is required;

Q.7 If we have a const based init for objects (array) then it does not make the object fully immutable;

<Stack>

<Heap>

`const obj = { name: "suresh" }`

but `obj.name = "sonu" // modify { allowed }`

Here const restrict reassignment of variable; not immutable

`const obj = {}`

`const obj = {}` X not allowed;

also `obj = { age: 20 }` X not allowed;

↳ new obj reassignment not allowed;
 is

New reference is not allowed in const ↳ new reference;

Q.7 So technically, with const objects are not fully immutable;

Q.8 How to make obj fully immutable?

Q.7 Yes we can make it fully immutable?

`Object.freeze()`

`Object.freeze();`

A.T.O

(i) `obj.freeze()`(ii) `obj.seal()`;

Using these two methods we can achieve immutability property;

(i) `Object.seal(obj)`:- restrict to add new key: pairs & delete existing key: value;↳ delete `obj.age` → false;`obj.phone` = --- Xbut you can update the existing ^{key of} value;eg `obj = {name: 'abc'}`, `Object.seal(obj)``obj.name = 'ritesh'` ✓`obj.age = 23` Xdelete `obj.name` X(ii) `Object.freeze(obj)`:- Here you even can't able to update the existing key: value;`obj = {name: 'abc'}`, `Object.freeze(obj)`;update X `obj.name = 'ritesh'` Xadd X `obj.age = 23` Xdelete X `delete obj.name` X

↳ This provides highest level of immutability, it creates a frozen obj;

⇒ `Object.isFrozen(obj)` $\begin{cases} T \\ F \end{cases}$ either frozen or not

`Object.isSealed(obj)` $\begin{cases} T \rightarrow \text{even frozen} \rightarrow \text{True} \\ F \rightarrow \text{for all rest cases;} \end{cases}$

① Half Seal properties :- `Object.preventExtensions(x)`;

Similar to seal but it allows to delete existing (key, value) pairs

Object.defineProperty: — you can add ^(control) prevention (configurable, not writable (readonly)) at key level;

Object.defineProperty(obj, 'name', {configurable: false});
obj.name = "ram" X it will not update;

So for frozen (internally it uses defineProperty());

Q7 {configurable: false} → you cannot able to delete;
you can pass multiple config {configurable: false,
writable: false}

Q. Can we make our own Object.seal()

Yes, How? using Object.preventExtension → similar to seal but allows delete

Object.defineProperty({config: false, writable: false})
↓
(It restrict from deletion)

make Object.freeze()

make Object.seal() ← true

code:- Function customSeal(obj) {
 let keys = Object.keys(obj);
 for (let i = 0; i < keys.length; i++) {
 Object.defineProperty(obj, keys[i], {configurable: false})
 }
 Object.preventExtension(obj)
}
if you need
writable: false
↓
freeze();